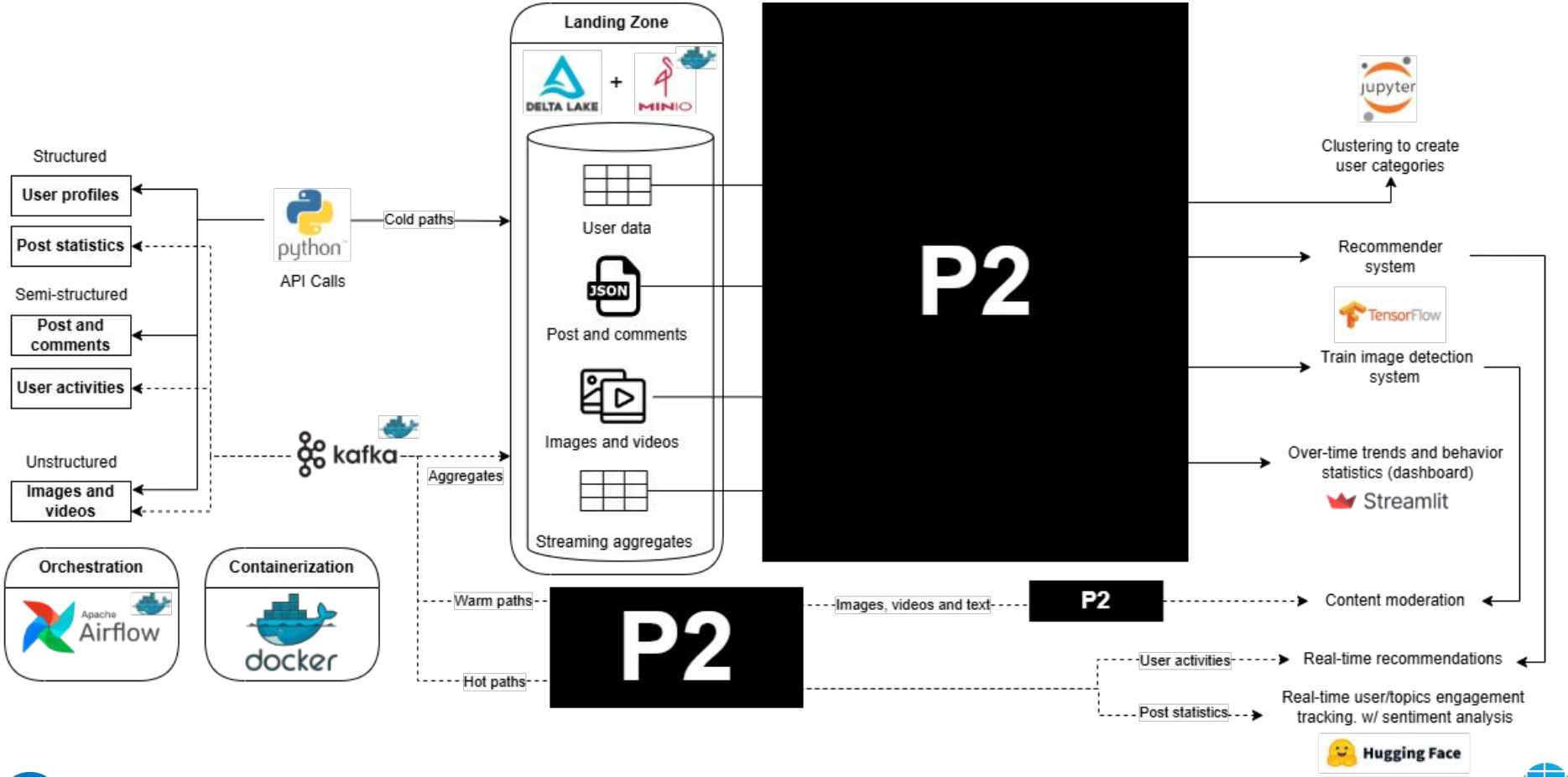
Big Data Management
Project P1

DTIM Research Group
Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Pipeline architecture

Pipeline architecture

- We explicitly ask you to define the **architecture** of your processing pipeline.


Landing Zone
DELTA LAKE
+
MINIO
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
Orchestration
Apache Airflow
Containerization
docker
Warm paths
Hot paths
P2
Images, videos and text
P2
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Content moderation
User activities
Real-time recommendations
Post statistics
Real-time user/topics engagement tracking. w/ sentiment analysis
Hugging Face

Pipeline architecture

- We explicitly ask you to define the **architecture** of your processing pipeline.
- The design you see is quite complex, and you should aim for something similar when defining the architecture.
- This is because you **do not need to implement everything** you present in the design.

Pipeline architecture

- We explicitly ask you to define the **architecture** of your processing pipeline.
- The design you see is quite complex, and you should aim for something similar when defining the architecture.
- This is because you **do not need to implement everything** you present in the design.

Constraint 1: Architecture diagram. You need to design a high-level diagram of your pipeline (you can take the one we propose as an example). It should include all the main stages of the architecture, how the data flows and, ideally, any technologies you use alongside other relevant aspects.

Contextualizing the problem

Contextualizing the problem

- Define the **domain**
  - Select a subject that excites you, but that also presents meaningful challenges.
  - You can be creative.

Contextualizing the problem

- Define the **domain**
  - Select a subject that excites you, but that also presents meaningful challenges.
  - You can be creative.

- Determine the **business value** (i.e. what problem(s) do you solve?)
  - Also, define **how** you plan on solving them.
  - Important: BDM is not a modeling/analysis course.

Contextualizing the problem

- Define the **domain**
  - Select a subject that excites you, but that also presents meaningful challenges.
  - You can be creative.

- Determine the **business value** (i.e. what problem(s) do you solve?)
  - Also, define **how** you plan on solving them
  - Important: BDM is not a modeling/analysis course.

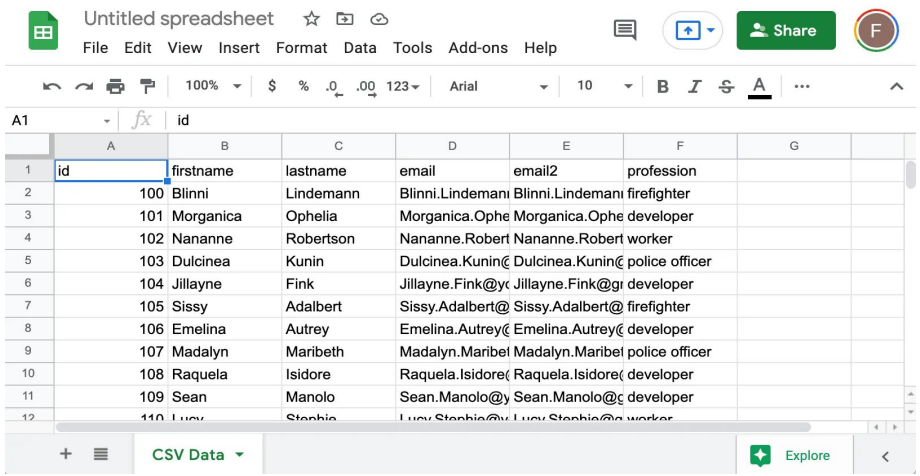
- Select the **data sources**
  - Take into consideration the **different data types** (structured, semi-structured and unstructured data). Understand what type of data you need.
  - You can create your own data.

Contextualizing the problem - Data types

Structured data

Customer_Details

Customer_id	Name	Address	Age
1	Billie	NY	22
2	Eilish	London	19
3	Ariana	Miami	18
4	Selena	New Jersey	32
5	Kety	Hawaii	42
6	Adele	Miami	29



	A	B	C	D	E	F
1	id	firstname	lastname	email	email2	profession
2	100	Blinni	Lindemann	Blinni.Lindemanı	Blinni.Lindemanı	firefighter
3	101	Morganica	Ophelia	Morganica.Ophe	Morganica.Ophe	developer
4	102	Nananne	Robertson	Nananne.Robert	Nananne.Robert	worker
5	103	Dulcinea	Kunin	Dulcinea.Kunin@	Dulcinea.Kunin@	police officer
6	104	Jillayne	Fink	Jillayne.Fink@y	Jillayne.Fink@gı	developer
7	105	Sissy	Adalbert	Sissy.Adalbert@	Sissy.Adalbert@	firefighter
8	106	Emelina	Autrey	Emelina.Autrey@	Emelina.Autrey@	developer
9	107	Madalyn	Maribeth	Madalyn.Maribet	Madalyn.Maribet	police officer
10	108	Raquela	Isidore	Raquela.Isidore(	Raquela.Isidore(	developer
11	109	Sean	Manolo	Sean.Manolo@y	Sean.Manolo@g	developer

Contextualizing the problem - Data types

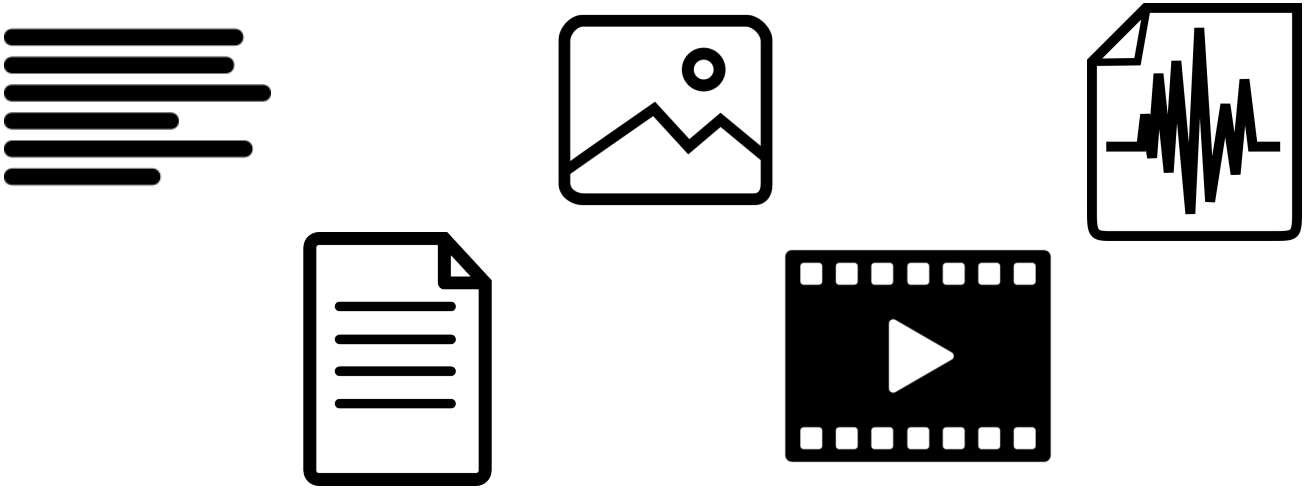
Semi-structured data

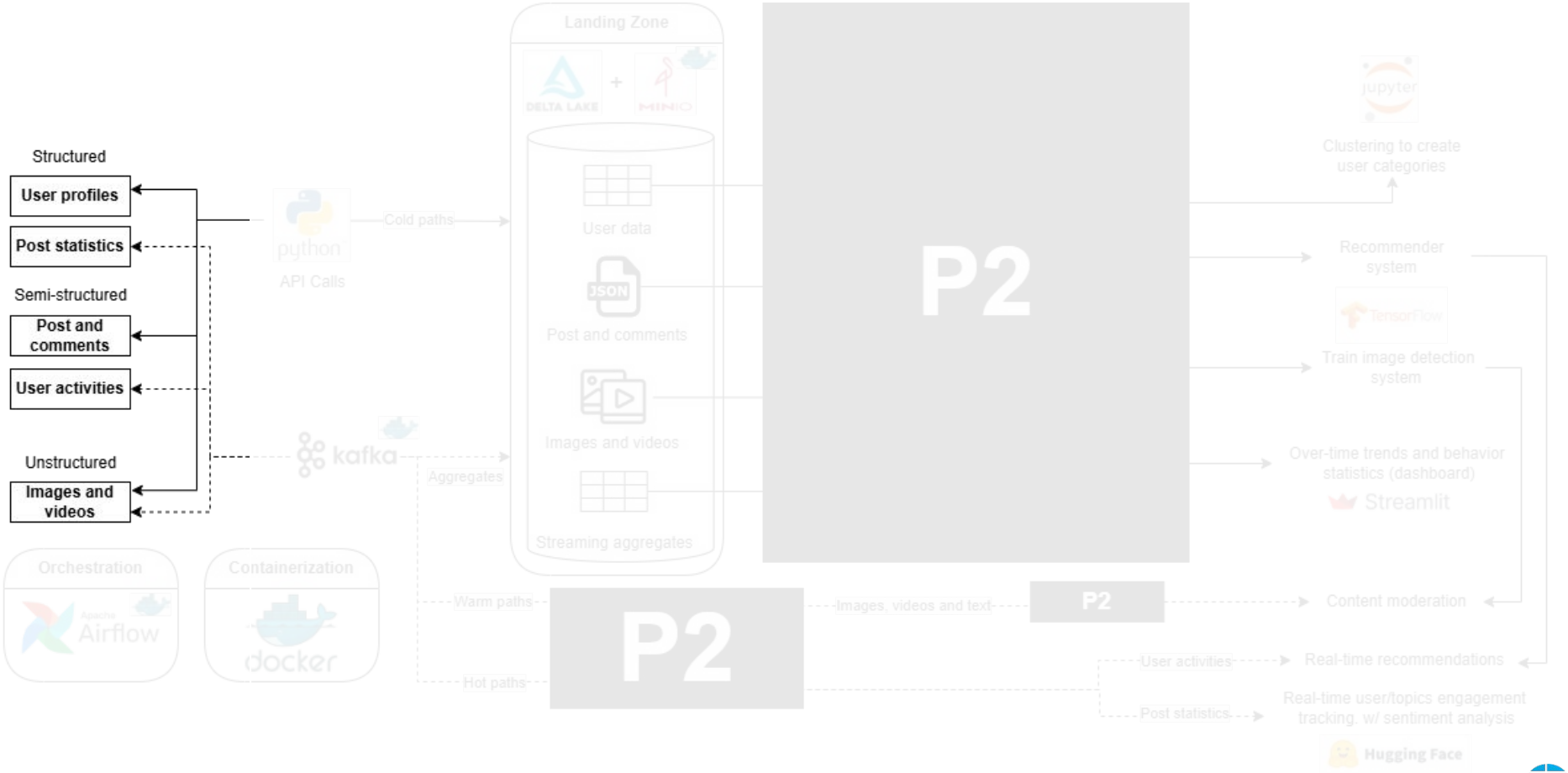

```
{
    "type": "message",
    "channel": "C2147483705",
    "user": "U2147483697",
    "text": "Hello world",
    "ts": "1355517523.000005",
    "is_starred": true,
    "pinned_to": ["C024BE7LT", ...],
    "reactions": [
        {
            "name": "astonished",
            "count": 3,
            "users": [ "U1", "U2", "U3" ]
        },
        {
            "name": "facepalm",
            "count": 1034,
            "users": [ "U1", "U2", "U3", "U4", "U5"
]
        }
```

```
<?xml version="1.0"?>
- <Dataset xmlns="http://www.safe.com">
  - <Building id="Surrey Head Office">
      <Address>"7445 132 St."</Address>
      <City>Surrey</City>
      <Province>BC</Province>
      <Country>Canada</Country>
    - <Location>
        <Longitude>-122.860</Longitude>
        <Latitude>49.138</Latitude>
      </Location>
      <Reference>https://www.google.ca/maps/
        3m1!4b1!4m5!3m4!1s0x5485dbd520cc
        122.8574636?hl=en</Reference>
    - <Room id="Admin_100">
        <Name>Reception</Name>
        <Category>Admin</Category>
        <Area units="m2">12</Area>
      - <Wall id="Admin_100_WallA">
          <material>wood</material>
        </Wall>
```

Contextualizing the problem - Data types

Unstructured data




Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
Landing Zone
DELTA LAKE
MINIO
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Orchestration
Apache Airflow
Containerization
docker
Warm paths
Hot paths
P2
Images, videos and text
P2
Content moderation
User activities
Real-time recommendations
Post statistics
Real-time user/topics engagement tracking, w/ sentiment analysis
Hugging Face

Contextualizing the problem

Constraint 2: Contextualization. Provide an adequate contextualization of the project, defining the domain, the task to tackle and the value you provide.

Constraint 3: Data selection. Identify what type of data you need. Then, search on popular repositories to obtain the required assets (or create them). Provide a brief explanation of each dataset. Note that you must include structured, semi-structured and unstructured data.

Contextualizing the problem

Factors that will **positively contribute** towards the grade:
- Employing several data sources (e.g. fetching data from different APIs) and/or several datasets (e.g. given a social media API, that you fetch both user profiles and posts).
- Great contextualization of the project (i.e. well thought-out domain and business value selection).
- Using “Big” Data.

Data Ingestion

Data Ingestion

- **Acquiring raw data** from various sources and **transferring** it into a system where it can be processed, stored and analyzed.

- Two main types

Data Ingestion - Batch

Data Source

country_data_01_02_2026.csv

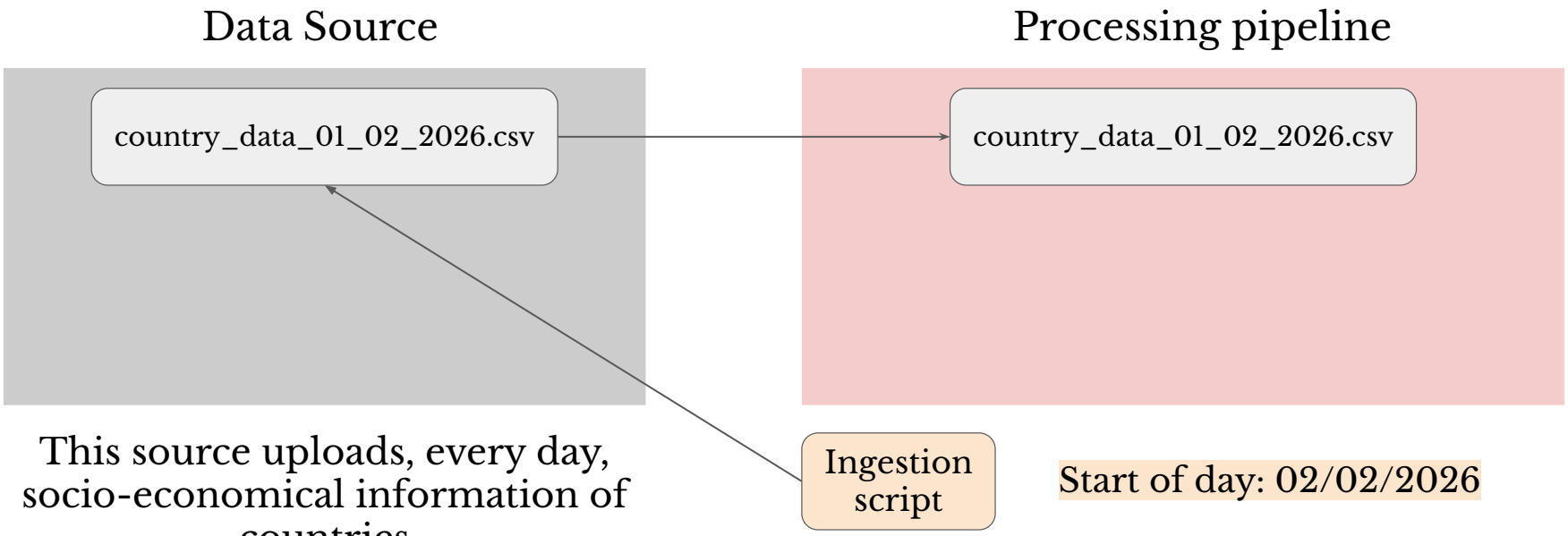
This source uploads, every day, socio-economical information of countries

Processing pipeline

Ingestion script

Start of day: 02/02/2026

Data Ingestion - Batch



This source uploads, every day, socio-economical information of countries

Start of day: 02/02/2026

Data Ingestion - Batch

Data Source

country_data_01_02_2026.csv

country_data_02_02_2026.csv

This source uploads, every day, socio-economical information of countries

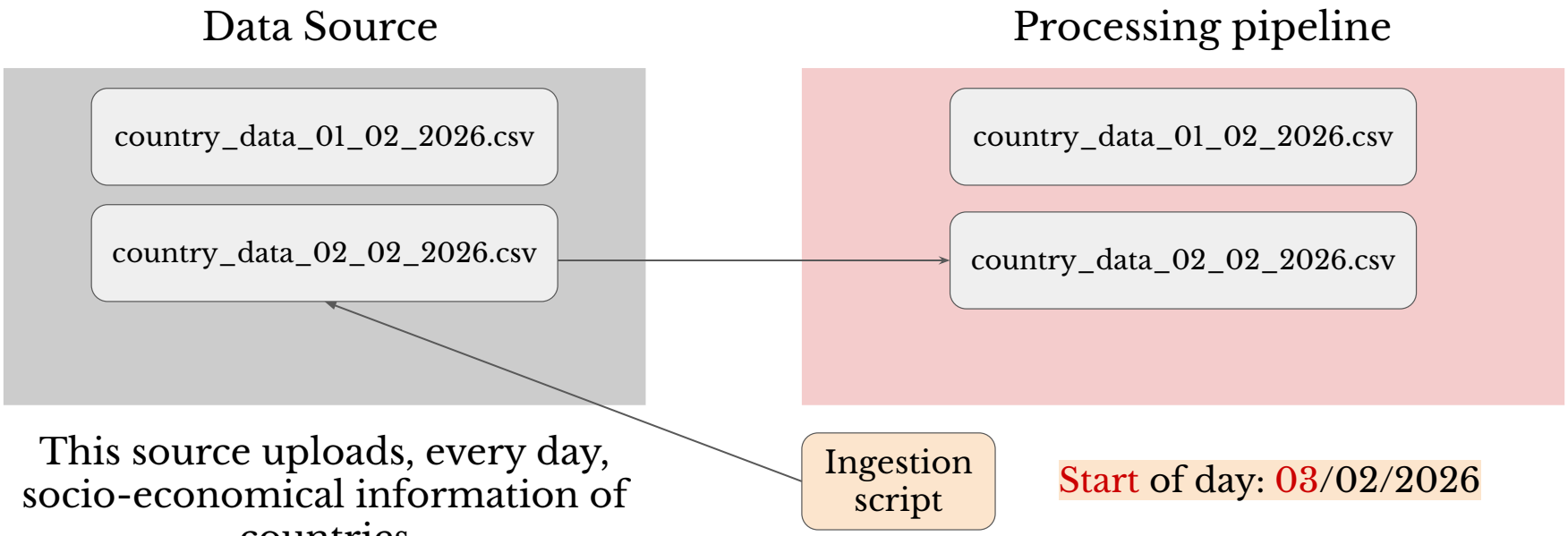
Processing pipeline

country_data_01_02_2026.csv

Ingestion script

End of day: 02/02/2026

Data Ingestion - Batch



This source uploads, every day, socio-economical information of countries

Start of day: 03/02/2026

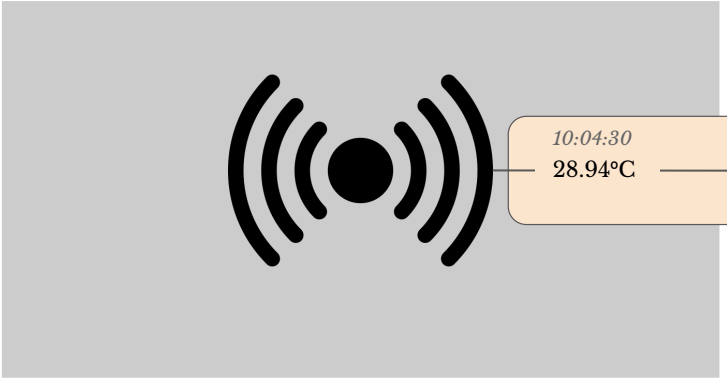
Data Ingestion - Streaming

Data Source

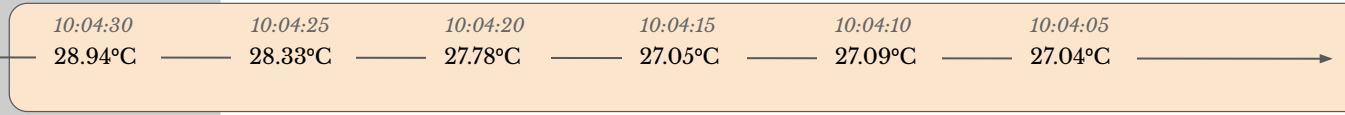
Temperature sensor that provides continuous (latency = 5 seconds) temperature values

Data Ingestion - Streaming

Data Source



Ingestion mechanism



Temperature sensor that provides continuous (latency = 5 seconds) temperature values

Data Ingestion

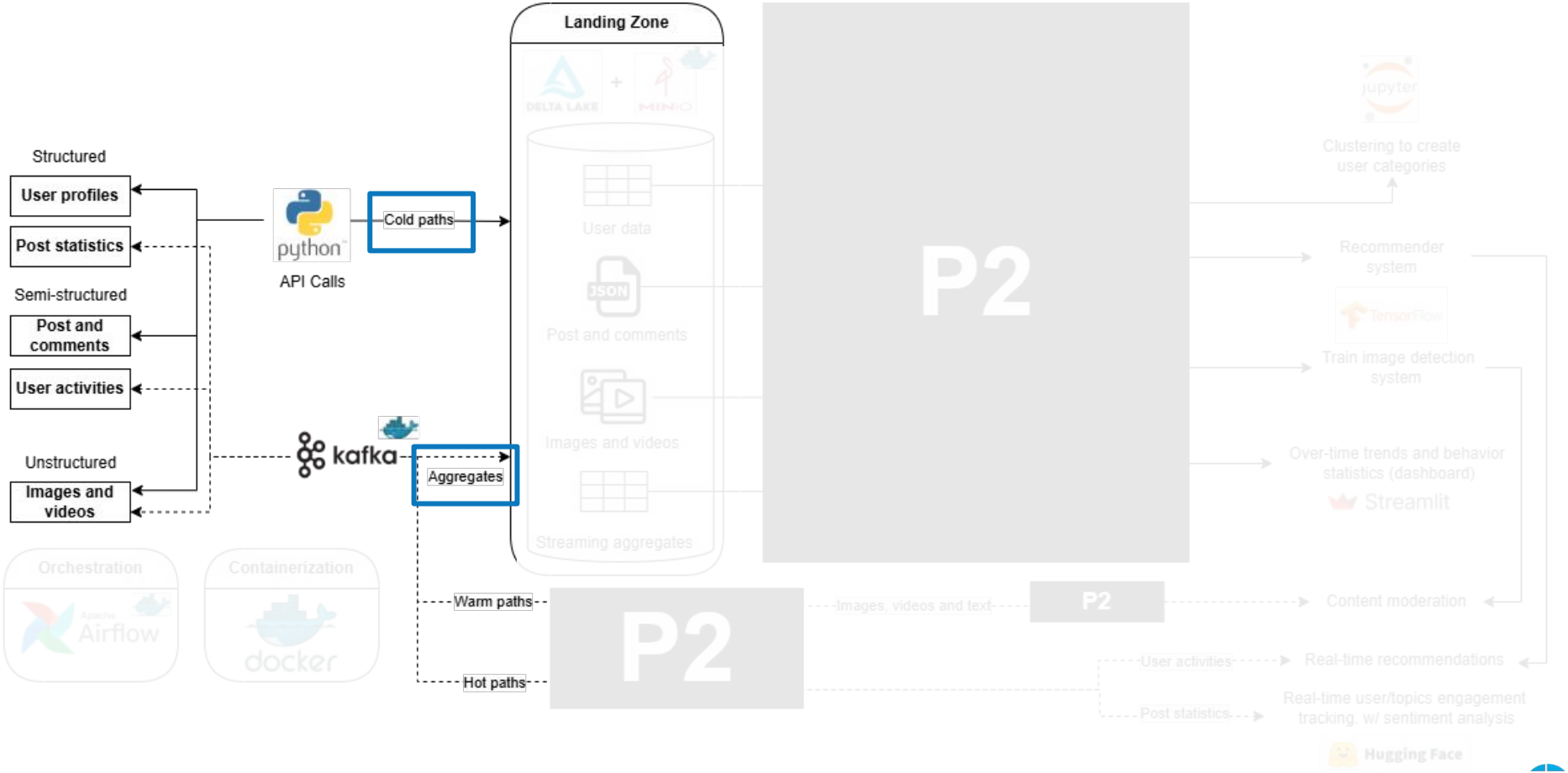
- **Acquiring raw data** from various sources and **transferring** it into a system where it can be processed, stored and analyzed.

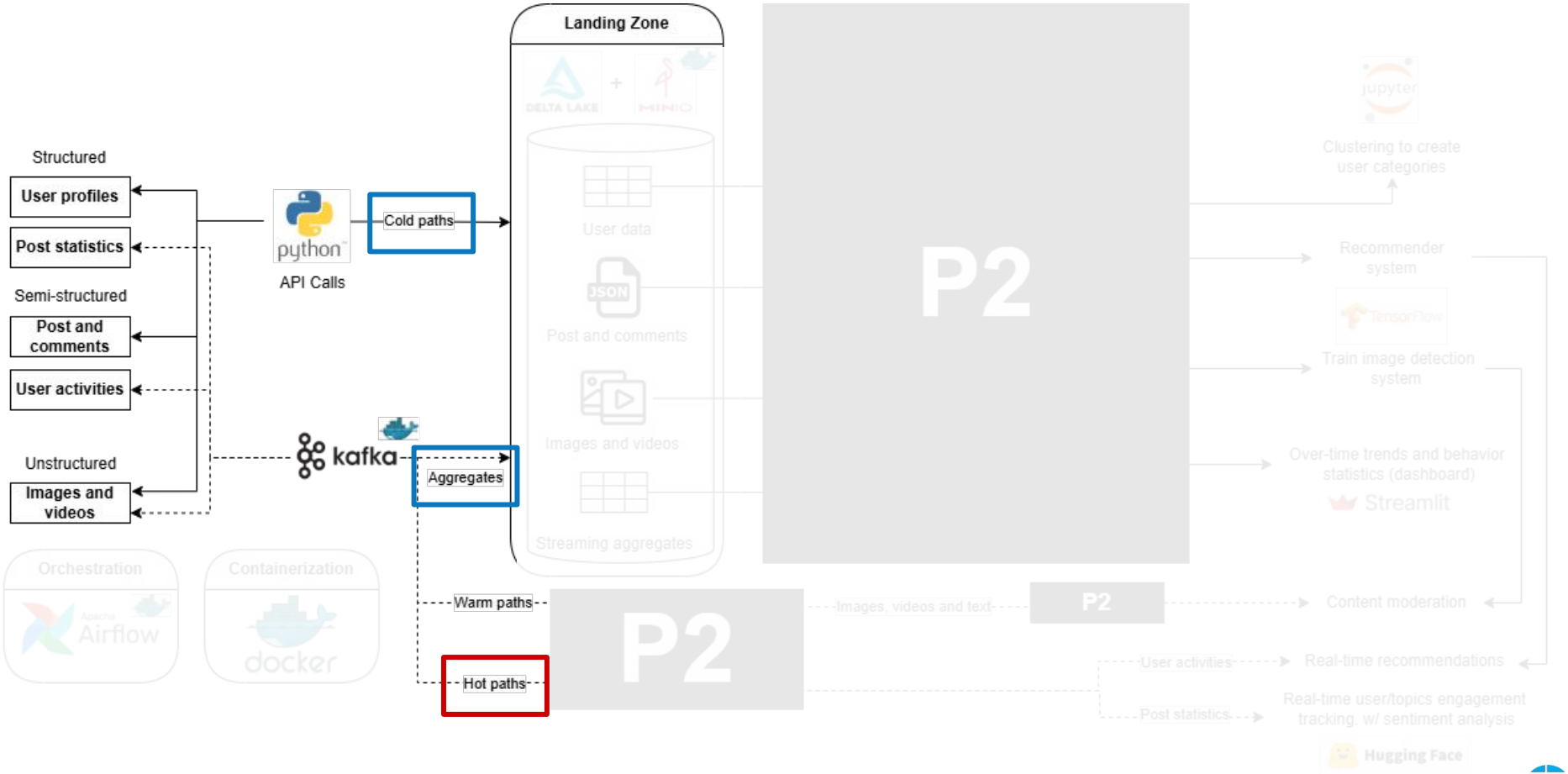
- Two main types:
  - **Batch ingestion**: data is collected in fixed-size chunks (batches), often at scheduled time intervals.
  - **Streaming ingestion**: data is continuously ingested and processed as it arrives, enabling real-time analytics.

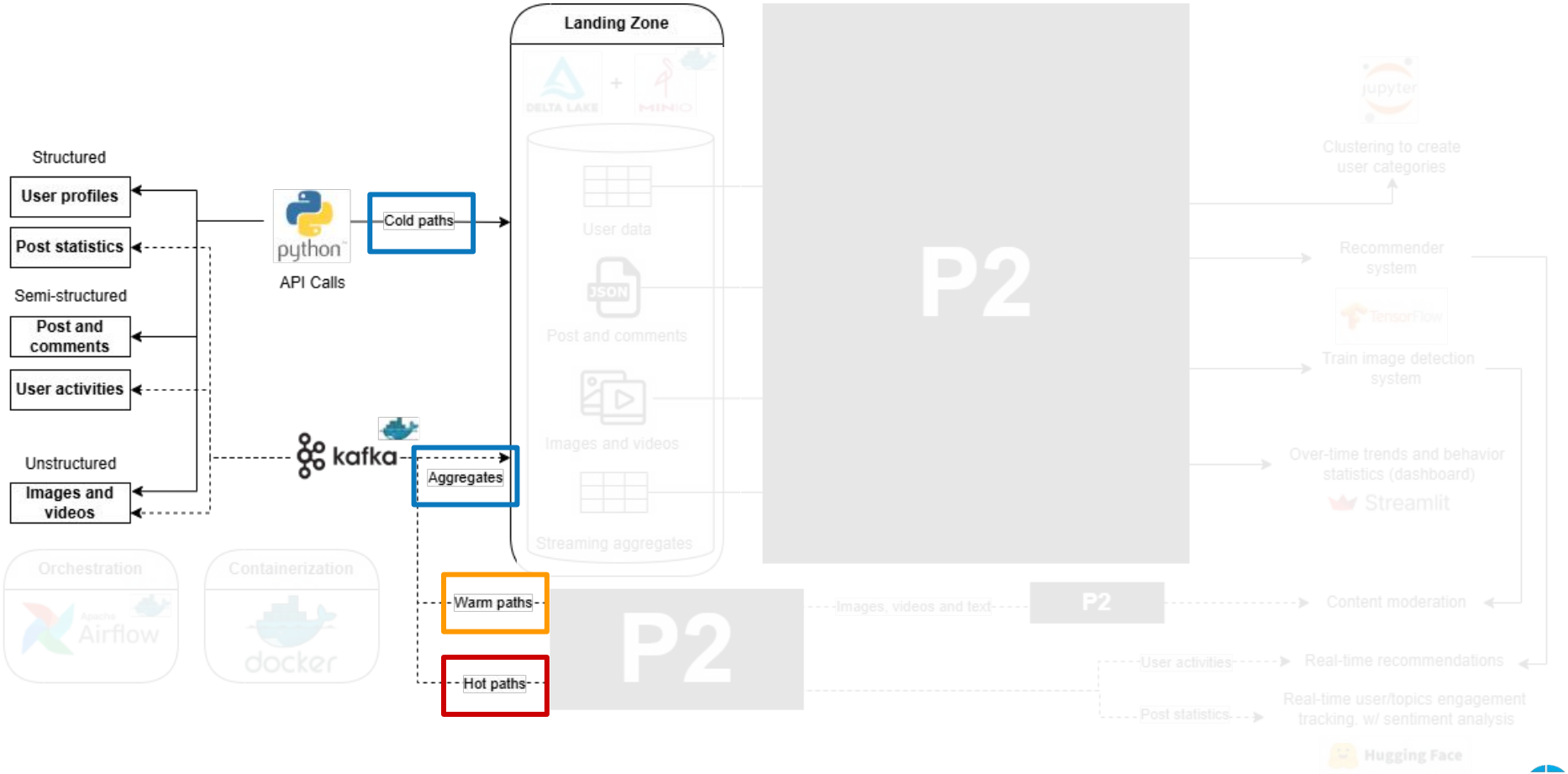
- Batch ingestion can be implemented with simple API calls via **Python** scripts. For streaming, we recommend **Apache Kafka**.

Data Ingestion - Processing paths

- Most complex systems combine **both** ingestions systems.
- By doing so, several **processing paths** can be defined


Landing Zone
DELTA LAKE
MINIO
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
Warm paths
Hot paths
P2
Images, videos and text
P2
Orchestration
Apache Airflow
Containerization
docker
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Content moderation
User activities
Real-time recommendations
Post statistics
Real-time user/topics engagement tracking, w/ sentiment analysis
Hugging Face


Landing Zone
DELTA LAKE
MINIO
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
Warm paths
Hot paths
P2
Images, videos and text
P2
Orchestration
Apache Airflow
Containerization
docker
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Content moderation
User activities
Real-time recommendations
Post statistics
Real-time user/topics engagement tracking, w/ sentiment analysis
Hugging Face


Landing Zone
DELTA LAKE
MINIO
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
Warm paths
Hot paths
P2
Images, videos and text
P2
Orchestration
Airflow
Containerization
docker
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Content moderation
User activities
Real-time recommendations
Post statistics
Real-time user/topics engagement tracking, w/ sentiment analysis
Hugging Face

Data Ingestion - Processing paths

- Most complex systems combine **both** ingestions systems.

- By doing so, several **processing paths** can be defined:
  - **Hot Path** (Real-Time Processing): This path is dedicated to handling time-sensitive data that requires immediate action.
  - **Cold Path** (Batch Processing): This approach handles data that do not need to be processed immediately, often obtained through scheduled batch jobs.
  - **Warm Path** (Near-Real-Time Processing): Positioned between the hot and cold paths, the warm path processes data that is not necessarily urgent but still requires timely analysis.

- You need to define the most appropriate paths for your case.

Data Ingestion

- It is **not acceptable** to ingest data manually.
  - You have to, at least, automate introducing the files into the pipeline.

Data Ingestion

- It is **not acceptable** to ingest data manually.
  - You have to, at least, automate introducing the files into the pipeline.

- A better alternative is to extract data from APIs, databases, web scraping, etc.
  - That is, directly interacting with **external systems**.

Data Ingestion

Constraint 4: Data ingestion design. Select an ingestion strategy (batch, streaming or both) for each of the data sources you have selected. This should be in accordance with the type of processing (i.e. path) that each data should undergo to fulfill the desired goals.

Constraint 5: Data ingestion implementation. Choose appropriate tools to automate / manage ingestion and implement your ingestion tasks.

Data Ingestion

Factors that will **positively contribute** towards the grade:
- Implementing both batch and stream processing pipelines.
- Defining several processing paths.
- Ingestion systems that communicate with external sources (i.e. that do not read files from your system, but rather that access non-local environments).
- Scheduling data ingestion tasks (e.g. read batch data from an API every hour). More on this later on.

Landing Zone

Landing Zone

- Store all the **raw data** that was ingested into a **single repository**. Doing this provides many benefits:
  - The starting point of your processing paths is homogenized.
  - It ensures data availability, traceability and reproducibility.
  - It does not force data to adapt to a given model

Landing Zone

- Store all the **raw data** that was ingested into a **single repository**. Doing this provides many benefits:
  - The starting point of your processing paths is homogenized.
  - It ensures data availability, traceability and reproducibility.
  - It does not force data to adapt to a given model

- Note that not all data needs to be stored, such as that coming from streaming sources
  - You can opt for storing some aggregations or statistics for later analysis.

Landing Zone - Data Lakes

- Our Landing Zone is, essentially, a **Data Lake**.
  - It is a different storage paradigm, born from the needs of Big Data.
  - Meant to store raw, large-scale, unstructured data.

Data Lakes

	Data Warehouse	Data Lake
Data type	Strictly structured (relational) data	All types: structured, semi-structured and unstructured
Data processing	Model-first, read-later	Read-first, model-later
Data size	Smaller	Large
Users	Business analysts	Data scientists & engineers
Use cases	BI, reporting	ML, analysis

Data Lakes

	Data Warehouse	Data Lake
Storage cost	Expensive	Cheap
Query speed	Very fast	**Slower**
Governance	Easy to apply	Harder to manage (**data swamp**)
Examples	Snowflake, Redshift, BigQuery	S3, Azure, Google Cloud, HDFS

Data Lakes

aprenderbigdata.com

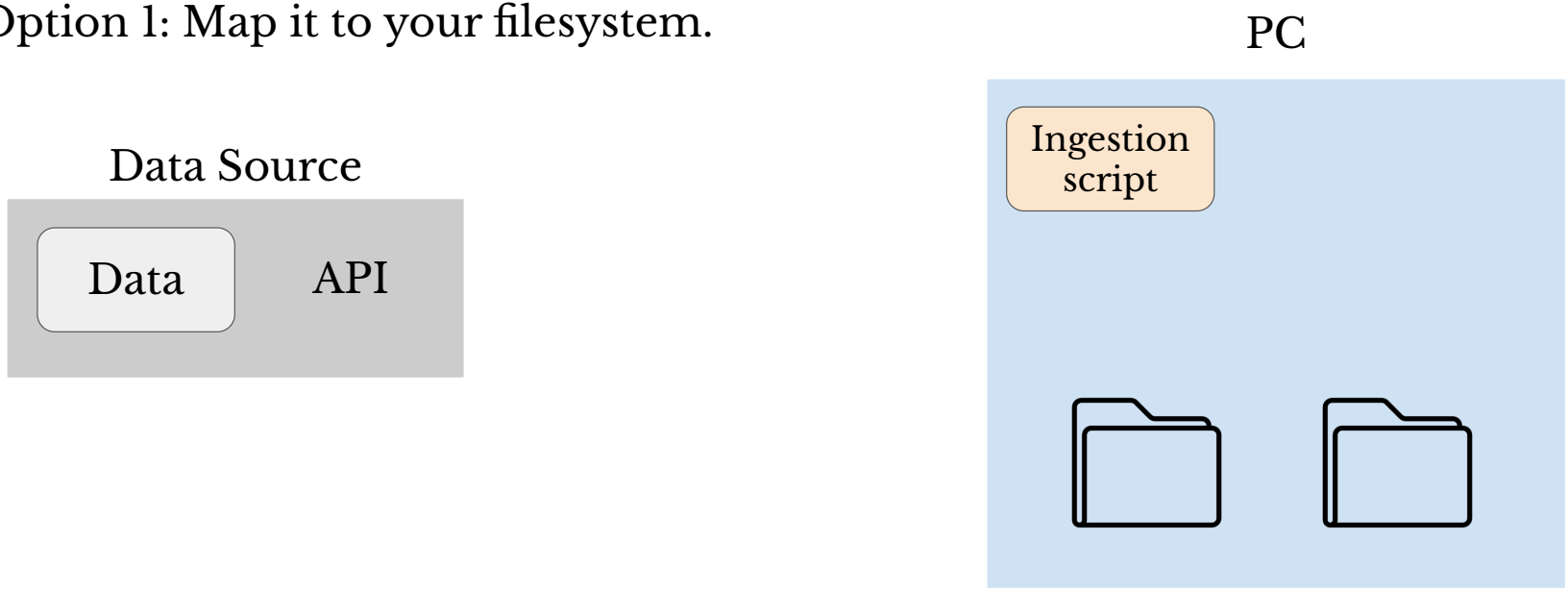
Landing Zone - Data Lakes

- Our Landing Zone is, essentially, a **Data Lake**.
  - It is a different storage paradigm, born from the needs of Big Data.
  - Meant to store raw, large-scale, unstructured data.

- How can we implement a Data Lake?

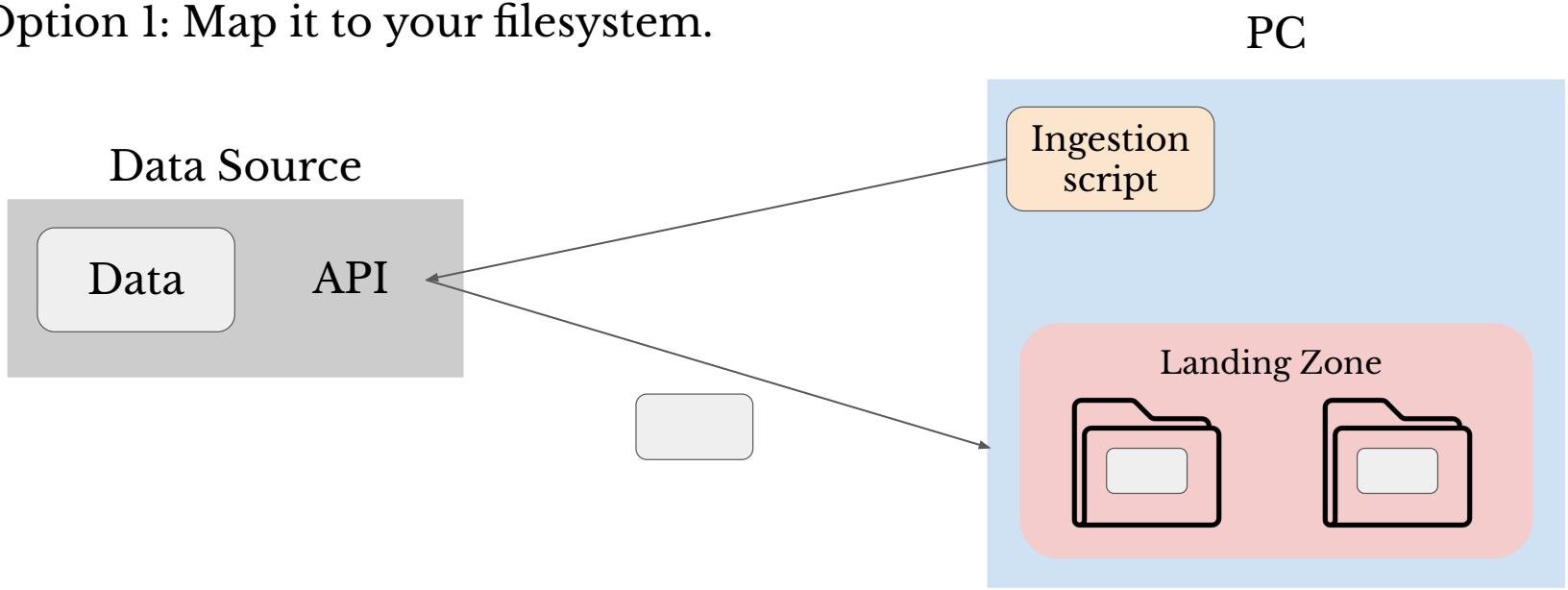
Landing Zone - Data Lakes

Option 1: Map it to your filesystem.



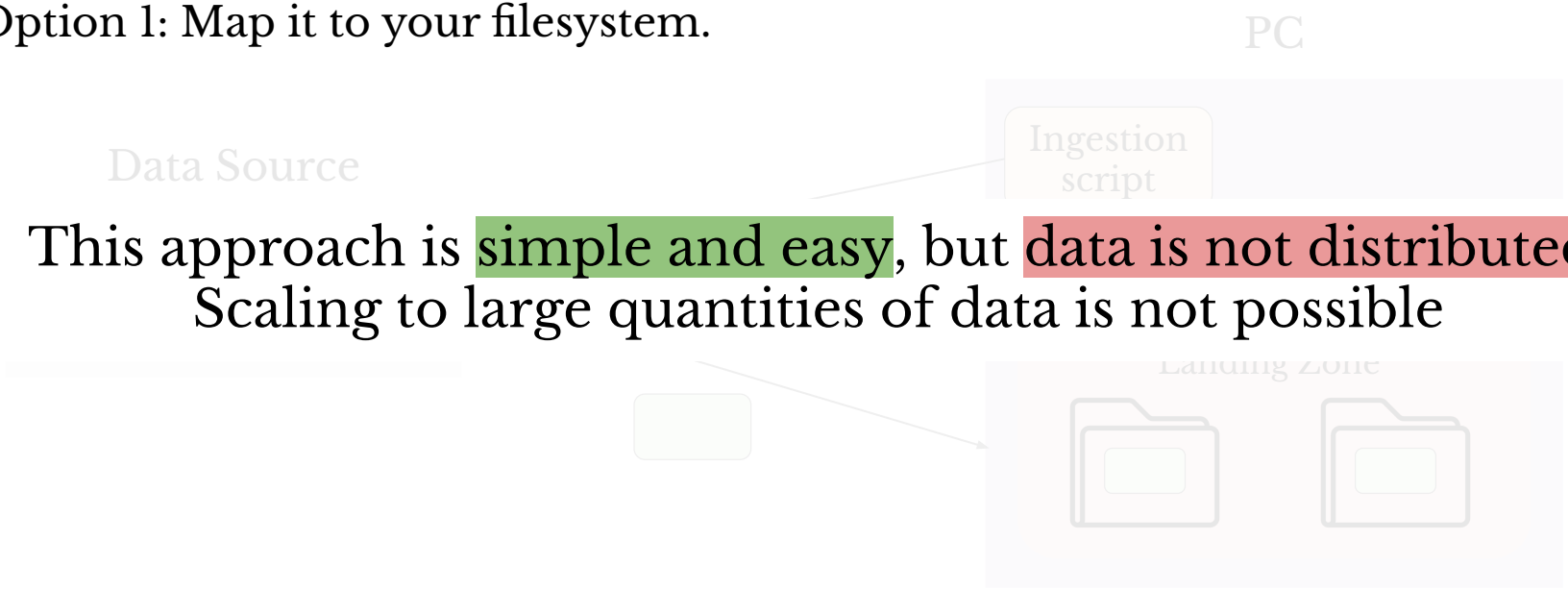
Landing Zone - Data Lakes

Option 1: Map it to your filesystem.



Landing Zone - Data Lakes

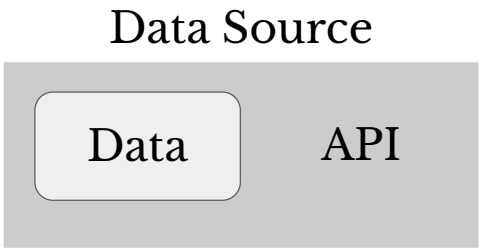
Option 1: Map it to your filesystem.

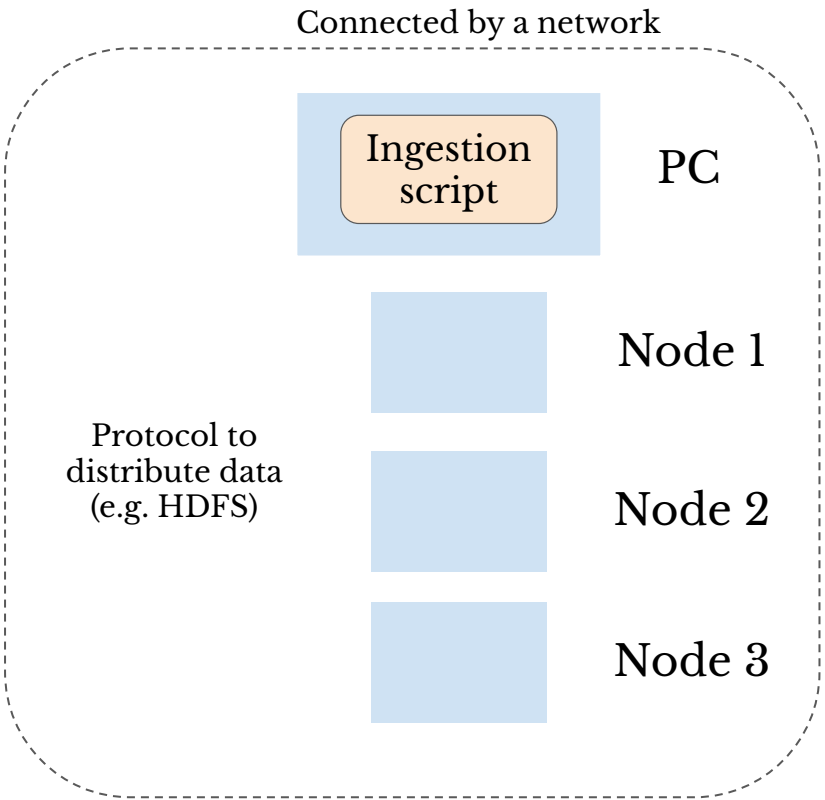


This approach is simple and easy, but data is not distributed.
Scaling to large quantities of data is not possible

Landing Zone - Data Lakes

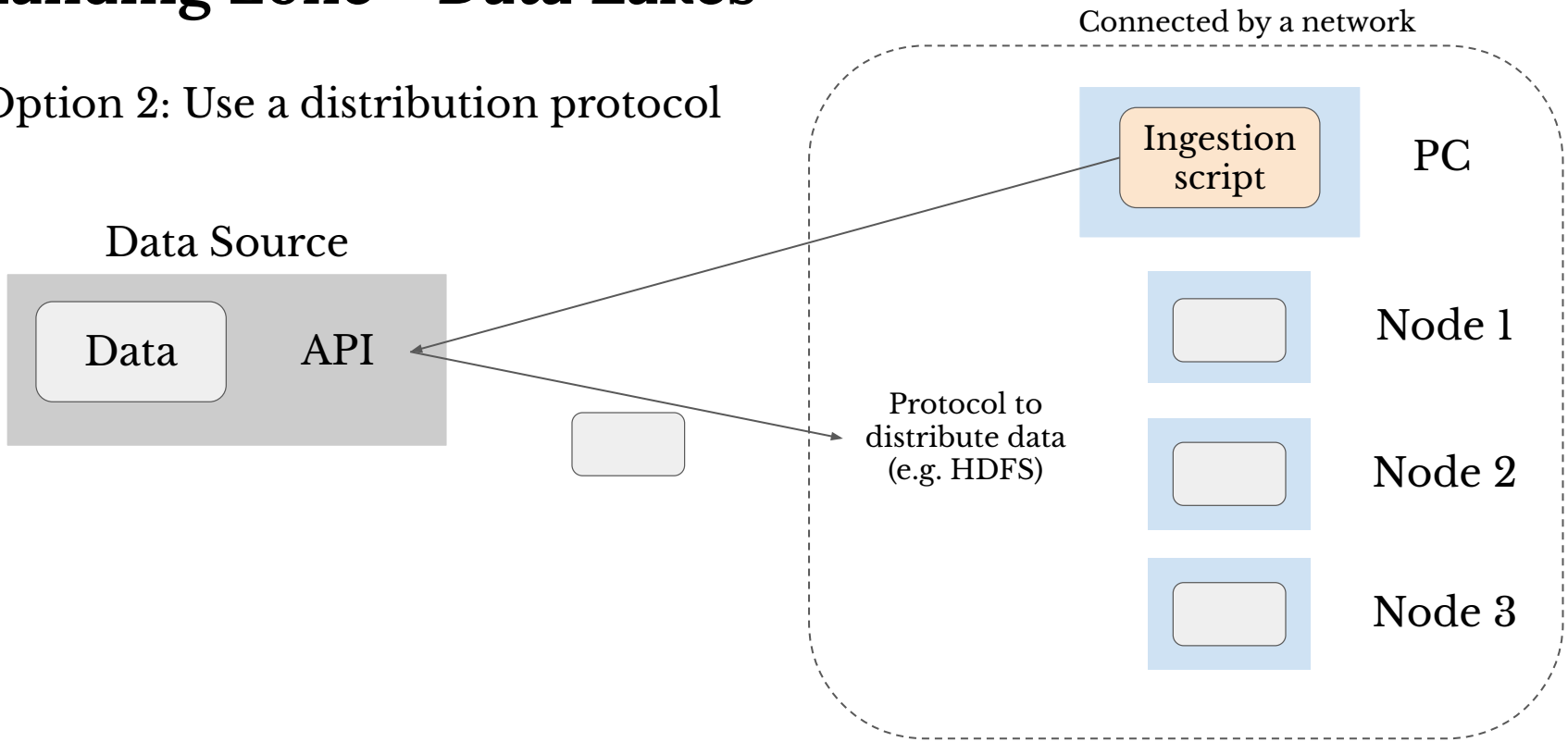
Option 2: Use a distribution protocol





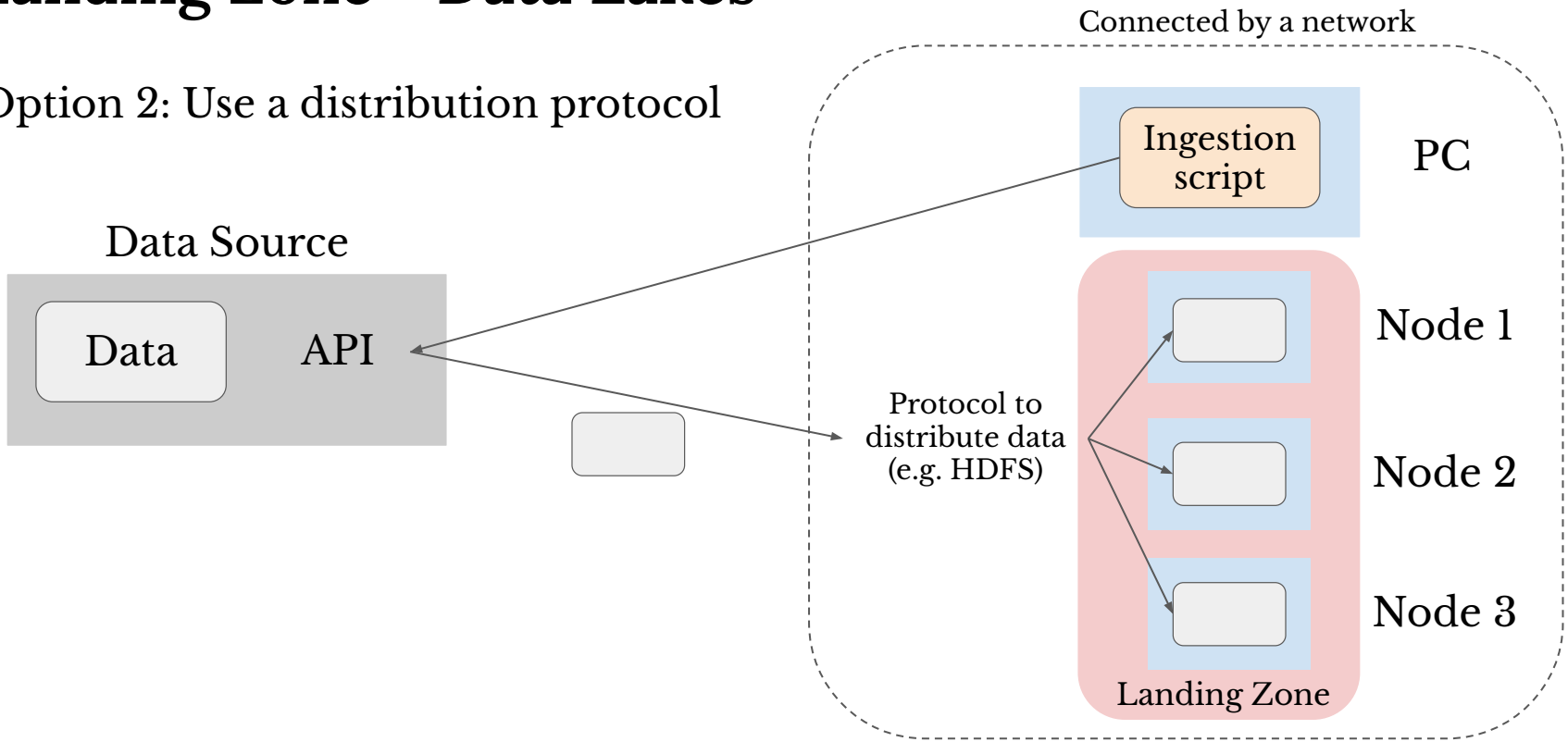
Landing Zone - Data Lakes

Option 2: Use a distribution protocol



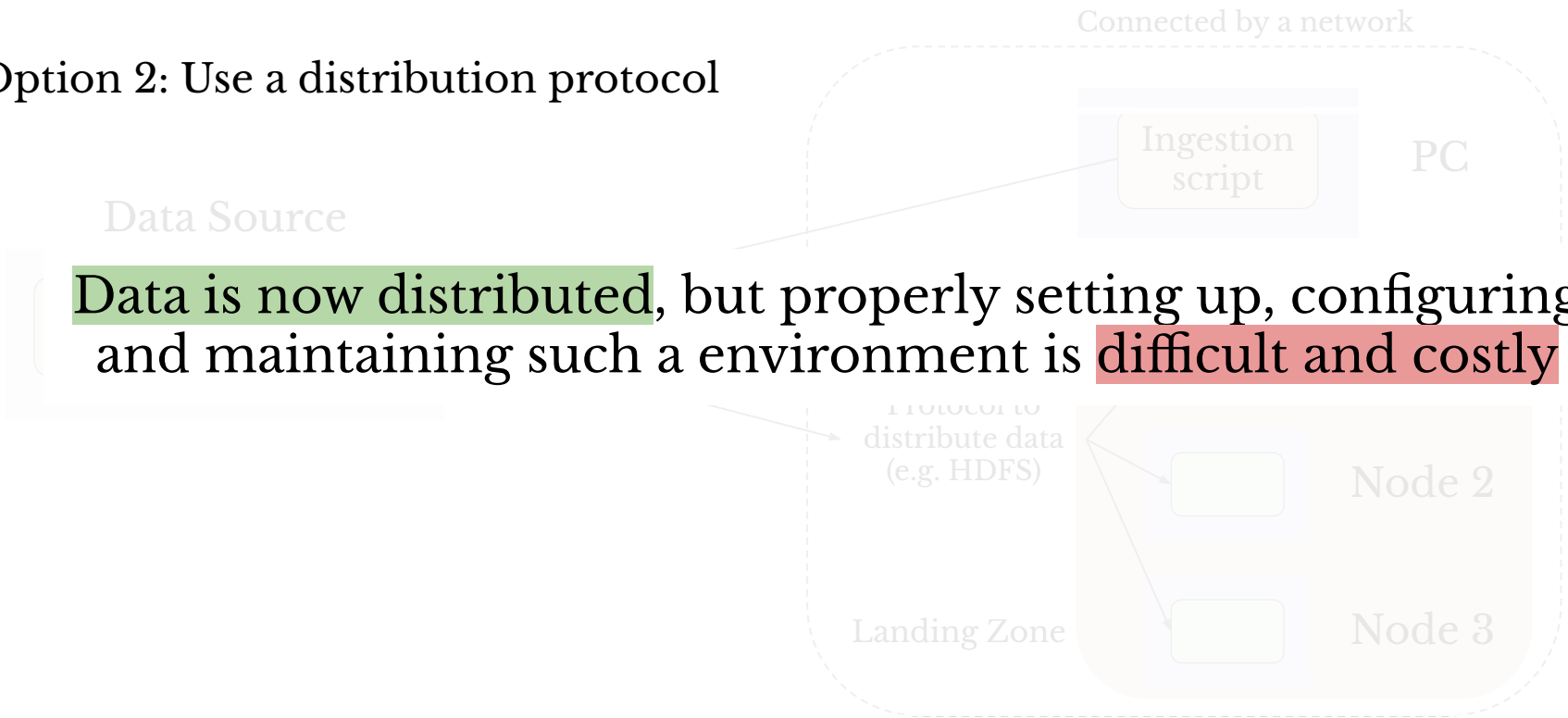
Landing Zone - Data Lakes

Option 2: Use a distribution protocol



Landing Zone - Data Lakes

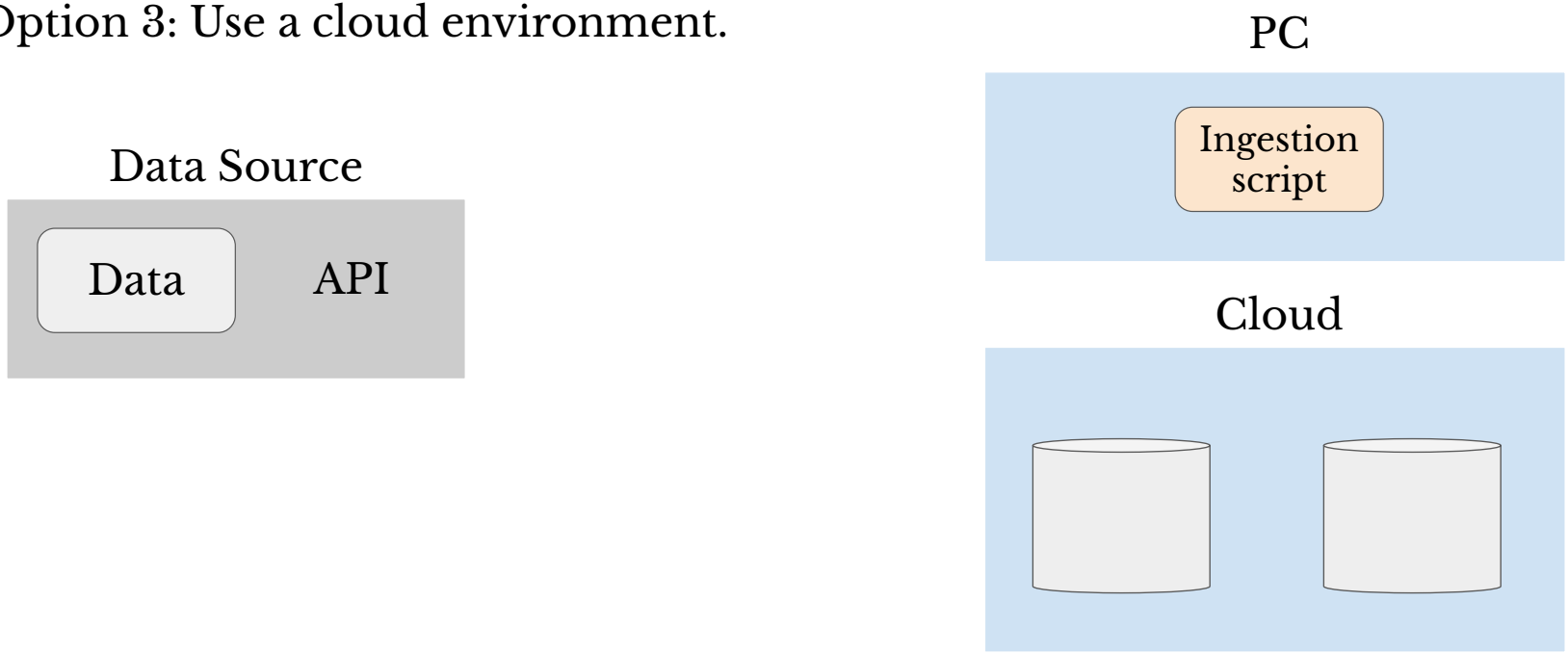
Option 2: Use a distribution protocol



Data is now distributed, but properly setting up, configuring and maintaining such a environment is difficult and costly

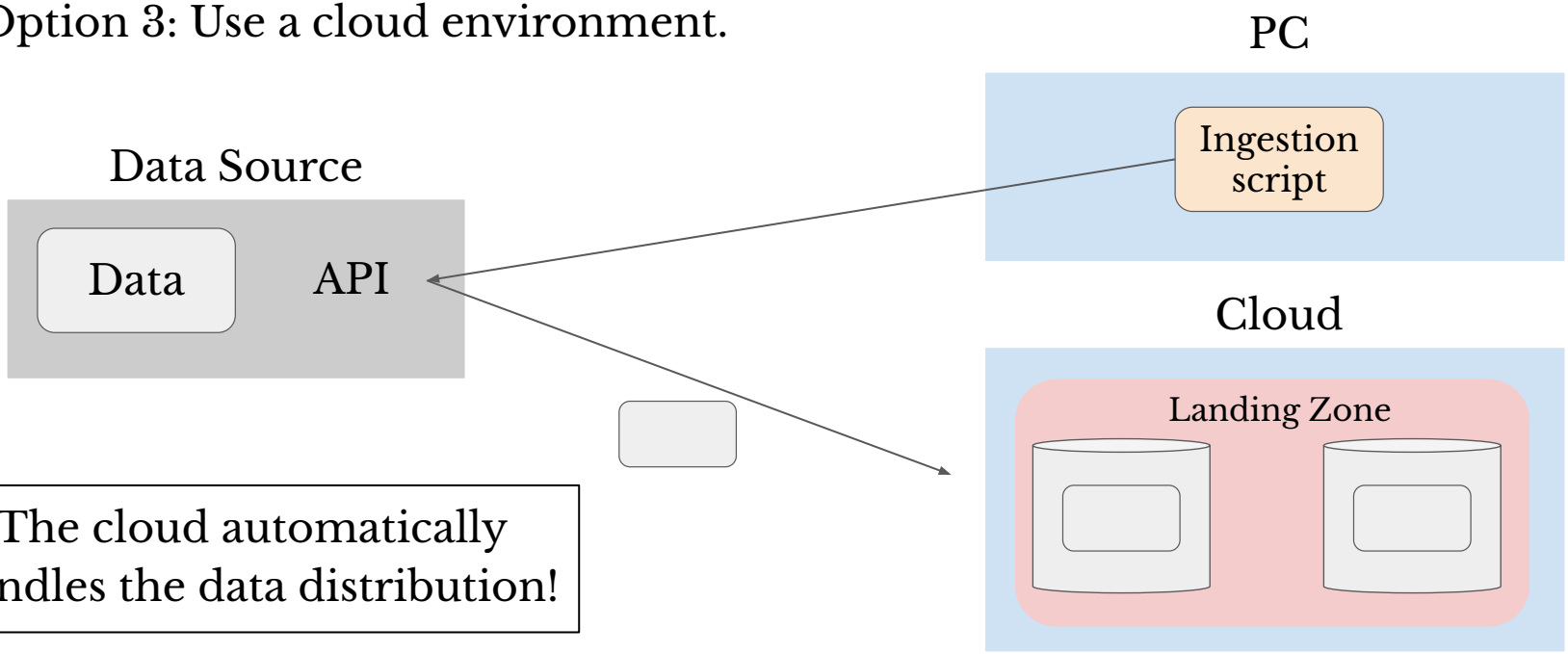
Landing Zone - Data Lakes

Option 3: Use a cloud environment.



Landing Zone - Data Lakes

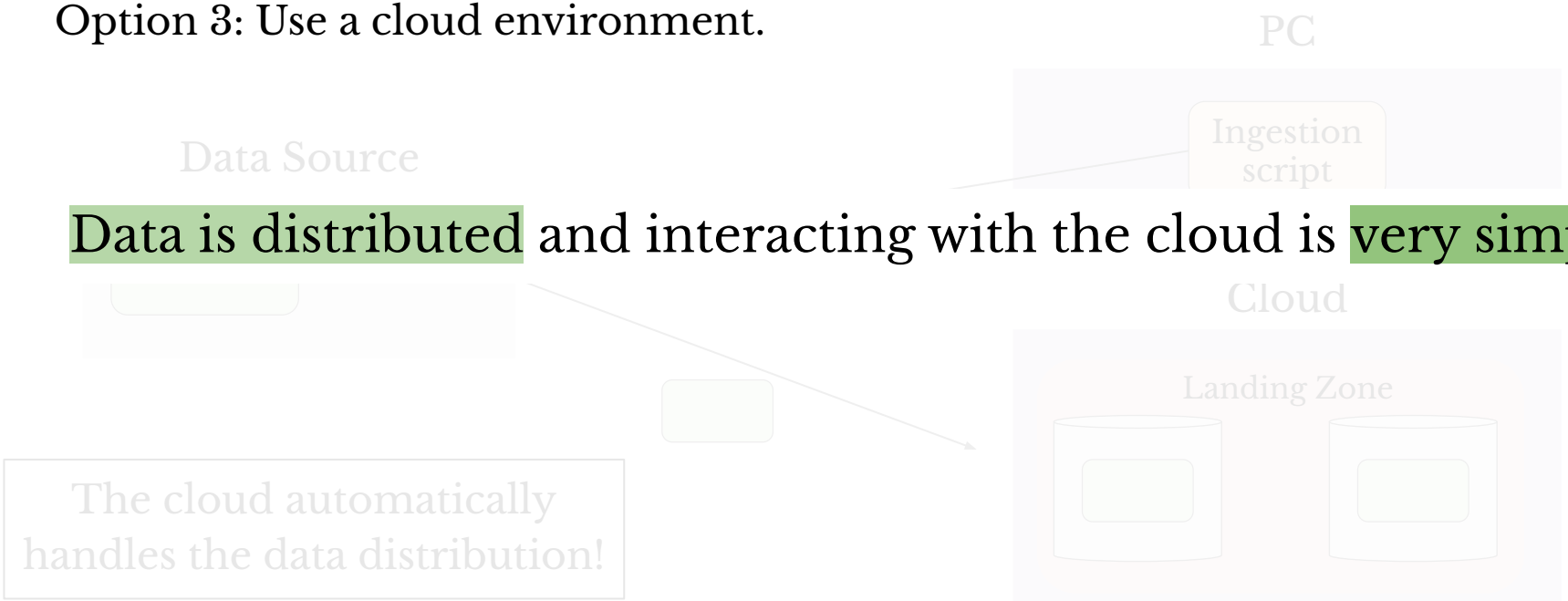
Option 3: Use a cloud environment.



The cloud automatically handles the data distribution!

Landing Zone - Data Lakes

Option 3: Use a cloud environment.



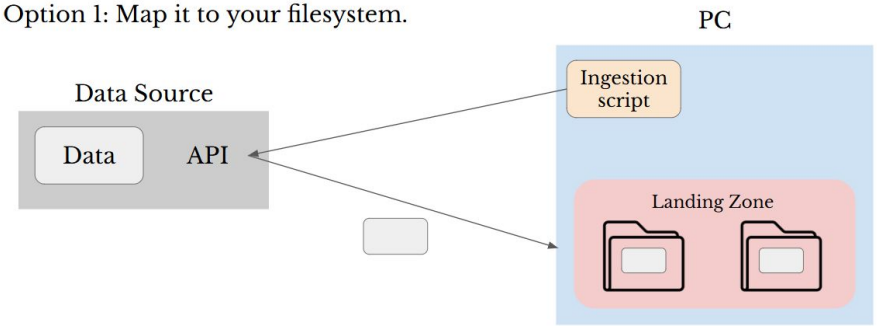
Data is distributed and interacting with the cloud is very simple

Landing Zone - Implementation

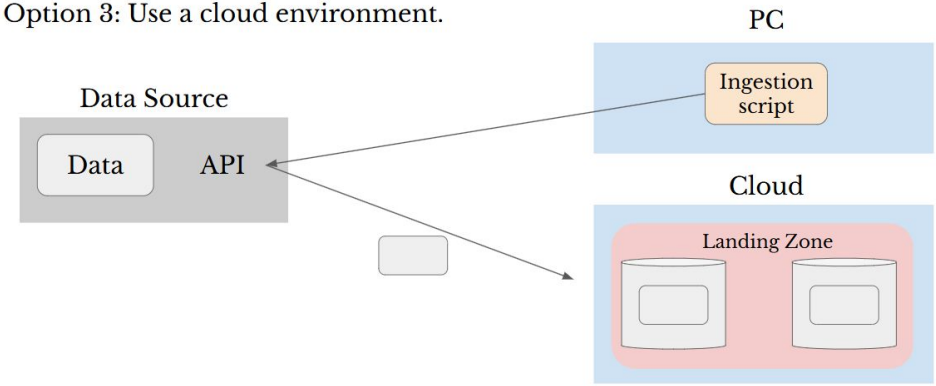
- From the data engineer perspective, option 1 and 3 are the same!
  - The only thing that changes is where the data is placed
  - Hence, **we recommend that you store the data locally**, knowing that, if distribution was needed, it could be easily switched up by cloud storage.
  - That is, treat this as a staging area.

Landing Zone - Object Stores

Option 1: Map it to your filesystem.

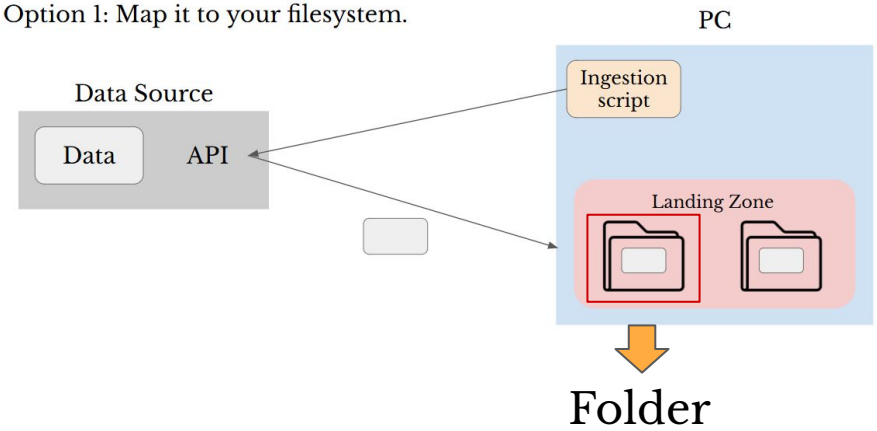


Option 3: Use a cloud environment.

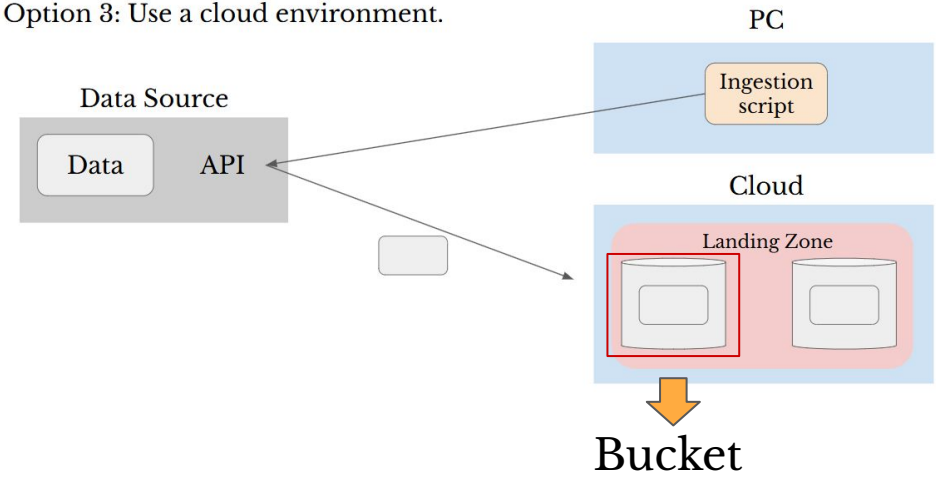


Landing Zone - Object Stores

Option 1: Map it to your filesystem.



Option 3: Use a cloud environment.



Object stores

- Folders are designed for **navigation and fast local access**.
  - File systems (like NTFS or ext4) organize files in a real hierarchy.
  - This hierarchy helps users and programs navigate easily

- Buckets are designed for **massive scale and distributed systems**.
  - Object stores (like Amazon S3) may store billions of objects across many servers.
  - A flat structure with buckets avoids the complexity and performance limits of deep hierarchies.
  - Buckets provide a simple top-level container and easy distribution across many machines

Object stores

- Folders are designed for **navigation and fast local access**.
  - File systems (like NTFS or ext4) organize files in a real hierarchy.
  - This hierarchy helps users and programs navigate easily

- Buckets are designed for **massive scale and distributed systems**.
  - Object stores (like Amazon S3) may store billions of objects across many servers.
  - A flat structure with buckets avoids the complexity and performance limits of deep hierarchies.
  - Buckets provide a simple top-level container and easy distribution across many machines

- PCs use folders, whereas **Object Stores** use buckets.

Object stores

- Object storages are a modern storage architecture that **manages data as discrete units called objects**, rather than as files.
  - That is, “elements” (e.g. videos, images, etc.) are stored in its **raw format** alongside internal metadata, which makes it easy to locate and retrieve.
  - This approach allows object storage to scale massively, store unstructured data efficiently and support features like versioning or replication.
  - Essentially, it is a **simple and effective way to store everything**.

Object stores

- Object storages are a modern storage architecture that **manages data as discrete units called objects**, rather than as files.
  - That is, “elements” (e.g. videos, images, etc.) are stored in its **raw format** alongside internal metadata, which makes it easy to locate and retrieve.
  - This approach allows object storage to scale massively, store unstructured data efficiently and support features like versioning or replication.
  - Essentially, it is a **simple and effective way to store everything**.

- We recommend using **MinIO**
  - Open-source and fully S3-compatible.
  - Easy programmatic interaction via the boto3 library
  - Simple UI for visual navigation

Landing Zone - Implementation

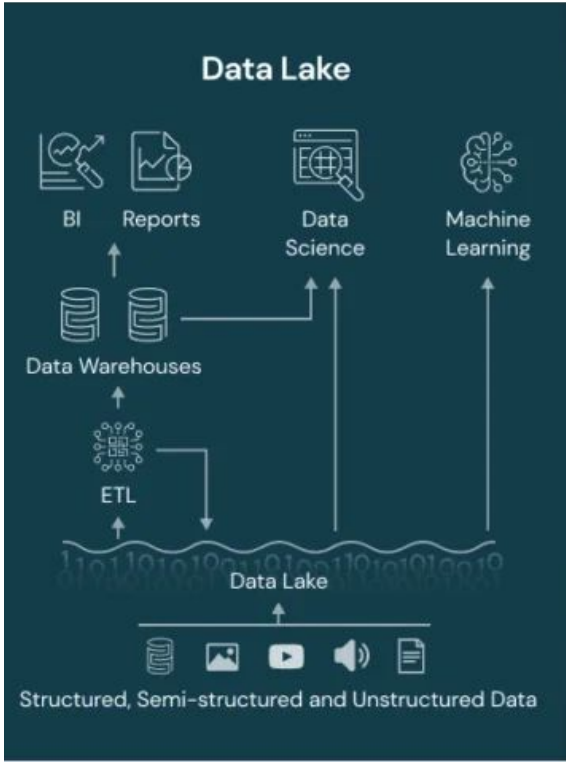
- From the data engineer perspective, option 1 and 3 are the same!
  - The only thing that changes is where the data is placed
  - Hence, **we recommend that you store the data locally**, knowing that, if distribution was needed, it could be easily switched up by cloud storage.
  - That is, treat this as a staging area.
- Use object stores

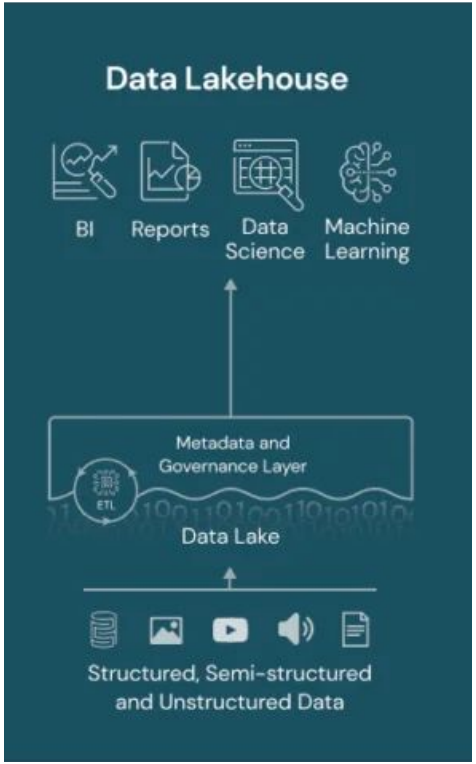
MINIO

Landing Zone - Data Lakehouse

- In recent years a new storage framework has emerged: the data lakehouse

Data Lakehouse





Landing Zone - Data Lakehouse

- In recent years a new storage framework has emerged: the data lakehouse:
  - These combine the storage philosophy of data lakes (large, heterogeneous repositories) with features designed to streamline data management and create an engineering experience similar to a data warehouse.

Landing Zone - Data Lakehouse

- In recent years a new storage framework has emerged: the data lakehouse:
  - These combine the storage philosophy of data lakes (large, heterogeneous repositories) with features designed to streamline data management and create an engineering experience similar to a data warehouse.

- To implement a data lakehouse we have to:
  - Store data in an object store (we have already seen this).
  - Introduce a governance & metadata layer on top of the data. This can be done via **Open Table Formats** (OTFs).

Landing Zone - Data Lakehouse

- OTFs are technologies that sit on top of object storage to manage large-scale data lake data as queryable database tables.
  - OTFs are, essentially, **metadata managers**. The handling of this metadata allows for advanced features such as those found in relational databases.

- For example, rather than storing *my_table.csv* as it is, I can use **Delta Lake** (an OTF) to create this structure:

```
/my_table/
    data/
        part-0001.parquet
        part-0002.parquet
    metadata/
        00001.metadata.json
```

Landing Zone - Implementation

- From the data engineer perspective, option 1 and 3 are the same!
  - The only thing that changes is where the data is placed
  - Hence, **we recommend that you store the data locally**, knowing that, if distribution was needed, it could be easily switched up by cloud storage.
  - That is, treat this as a staging area.
- Use object stores and OTFs

MINIO DELTA LAKE

Data Lakehouse

Landing Zone

- In the Landing Zone we store the raw data without transformations. However, we do apply some **organization and structuring**.
  - If not, a data lake becomes a data swamp.

Landing Zone

- In the Landing Zone we store the raw data without transformations. However, we do apply some **organization and structuring**.
  - If not, a data lake becomes a data swamp.

- To do so, you have to, at least, do the following:
  - Separate the incoming data based on some criteria (e.g. source). You can create these separations in a bucket.
  - Employ a naming convention.

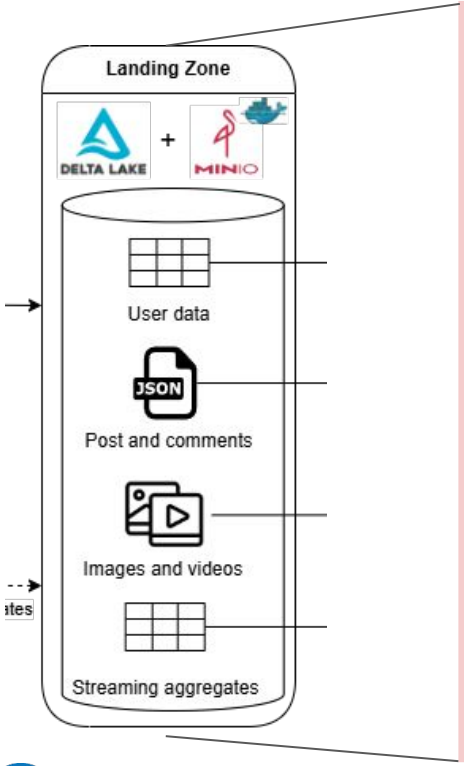
Landing Zone

- In the Landing Zone we store the raw data without transformations. However, we do apply some **organization and structuring**.
  - If not, a data lake becomes a data swamp.

- To do so, you have to, at least, do the following:
  - Separate the incoming data based on some criteria (e.g. source). You can create these separations in a bucket.
  - Employ a naming convention.

- Additional management techniques include those supported by OTFs or the extension of the pipeline with additional zones.

Landing Zone - Example



Landing Zone

User data

user_data/user_1/
data/
part-0001.parquet
part-0002.parquet
metadata/
metadata-01.json

user_data/user_2/
data/
part-0001.parquet
part-0002.parquet
metadata/
metadata-01.json

...

Media

media/
images/
image-0001/
data/image.png
metadata/metadata-01.json
image-0002/
data/image.png
metadata/metadata-01.json
...
videos/
video-0001/
data/video.mp4
metadata/
metadata-01.json
metadata-02.json
...

...

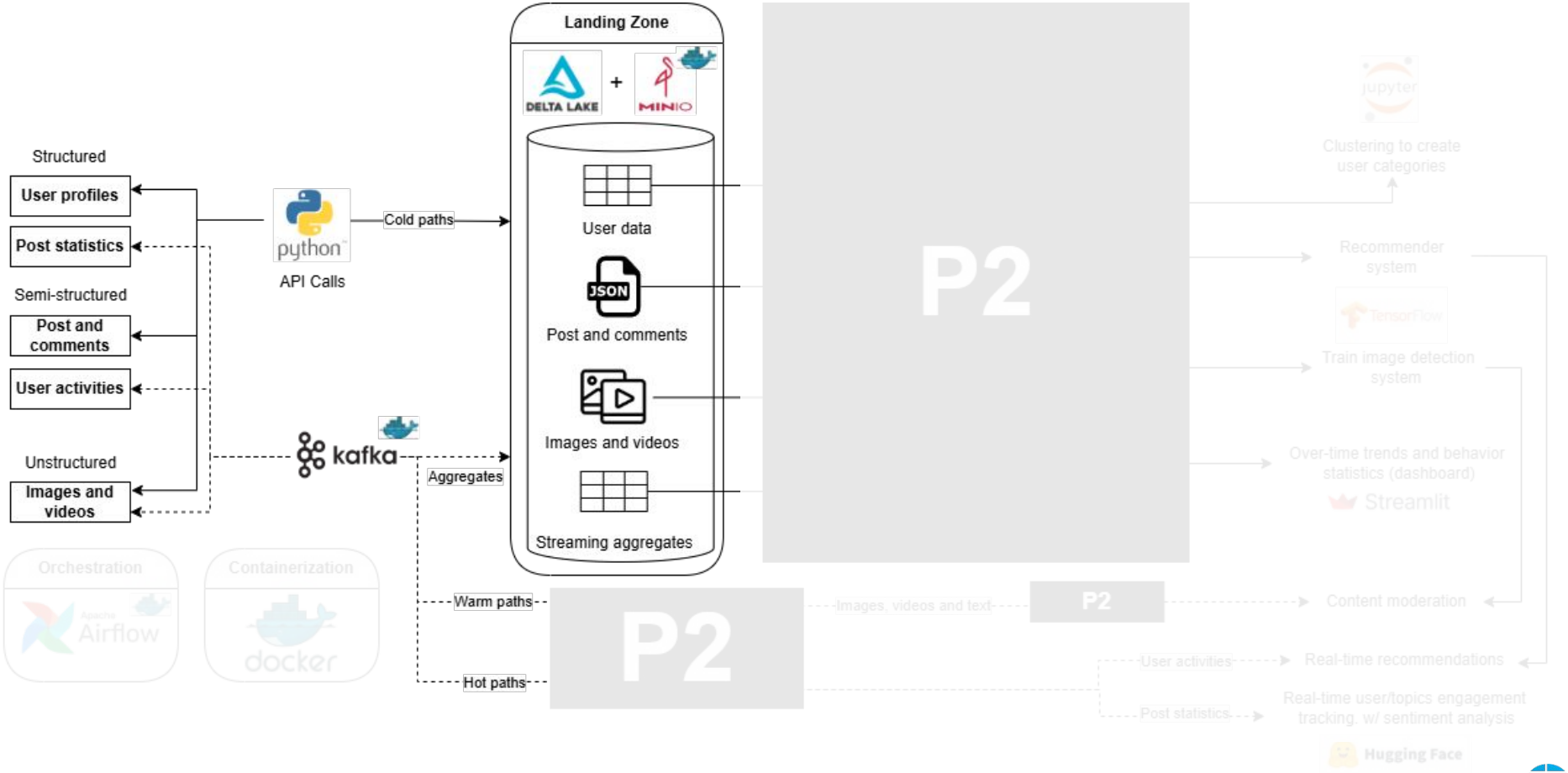
Landing Zone - Implementation

- From the data engineer perspective, option 1 and 3 are the same!
  - The only thing that changes is where the data is placed
  - Hence, **we recommend that you store the data locally**, knowing that, if distribution was needed, it could be easily switched up by cloud storage.
  - That is, treat this as a staging area.
- Use object stores and OTFs.

MINIO DELTA LAKE

Data Lakehouse

- Organize the data & apply a naming convention.
- Include additional management aspects.


Landing Zone
DELTA LAKE
+
MINIO
Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
Warm paths
Hot paths
P2
Images, videos and text
P2
Orchestration
Apache Airflow
Containerization
docker
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Content moderation
User activities
Real-time recommendations
Post statistics
Real-time user/topics engagement tracking, w/ sentiment analysis
Hugging Face

Landing Zone

Constraint 6: Landing Zone design. First, select an appropriate storage solution based on the nature of your data (e.g. data lakehouse). Second, define an organizational structure for storing raw data (e.g. different folders/buckets, naming convention, data partitioning). Third, indicate the series of technologies and methods to be employed.

Constraint 7: Landing Zone implementation. Choose appropriate tools to implement the Landing Zone, deploy it and store the data that was ingested.

Landing Zone

Factors that will **positively contribute** towards the grade:
- Using object storages.
- Defining a data lakehouse (use OTFs).
- In-depth metadata management.

Orchestration & Containerization

Orchestration

- The initial versions of the pipeline are meant to be developed asynchronously. Nonetheless these pipelines are **meant to be automated** (more precisely, orchestrated).

Orchestration

- The initial versions of the pipeline are meant to be developed asynchronously. Nonetheless these pipelines are **meant to be automated** (more precisely, orchestrated).

- Key aspects of workflow orchestration include:
  - **Task scheduling**: Defining when each process should run, whether on a periodic basis (e.g., daily, hourly) or triggered by specific events.
  - **Dependency management**: Ensuring that tasks are executed in the correct order
  - **Error handling and retries**: Automatically detecting failures, logging errors, and retrying failed tasks as necessary to ensure robustness.

- We recommend exploring **Apache Airflow**

Orchestration

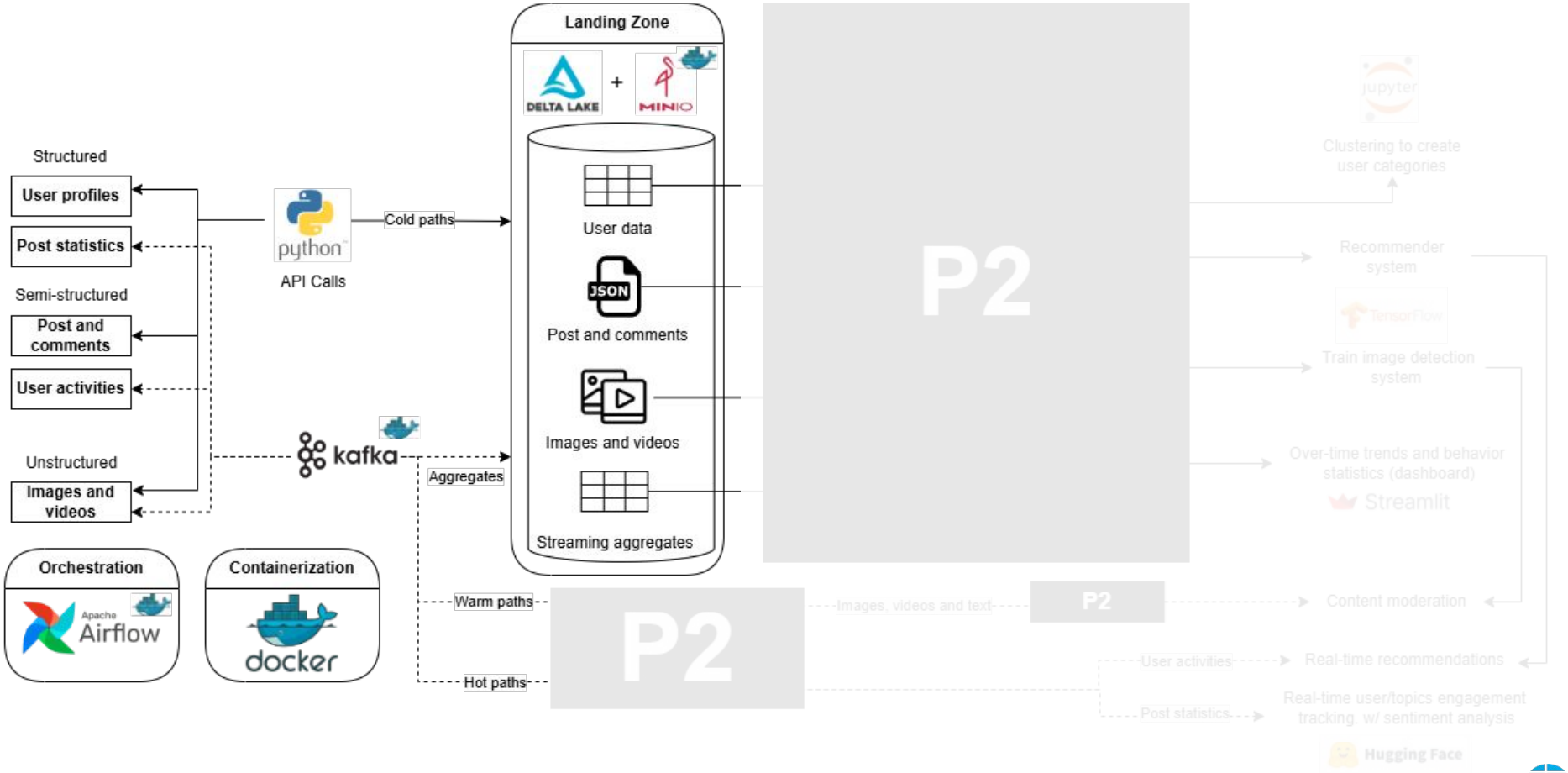
Constraint 8: Pipeline orchestration. Once the implementation for the ingestion and Landing Zone have been completed, you have to orchestrate the execution (i.e. automate the execution of the scripts so you do not have to manually trigger each task). Preferably, this should be done via Apache Airflow, although Python scripts are accepted as an alternative.

Containerization

- Some of the proposed tools (e.g. Apache Kafka) require the deployment of some service. You can do this manually, but an easier and more practical alternative is to use a **containerization tool** such as **Docker**.

- These tools allow you to **streamline the deployment of services**, and are ubiquitous in the current Data Science landscape.

Constraint 9: Containerization. If you employ any tool that requires the deployment of a service, you should do so via a containerization tool.


Landing Zone
DELTA LAKE
+
MINIO
Structured
User profiles
Post statistics
Semi-structured
Post and comments
User activities
Unstructured
Images and videos
python
API Calls
Cold paths
kafka
Aggregates
User data
JSON
Post and comments
Images and videos
Streaming aggregates
P2
Orchestration
Apache Airflow
Containerization
docker
Warm paths
Hot paths
P2
Images, videos and text
P2
User activities
Post statistics
jupyter
Clustering to create user categories
Recommender system
TensorFlow
Train image detection system
Over-time trends and behavior statistics (dashboard)
Streamlit
Content moderation
Real-time recommendations
Real-time user/topics engagement tracking, w/ sentiment analysis
Hugging Face

Deliverables

Main deliverable

- It has to implement:
  - Architecture design
  - Stage 0: project context
  - Stage 1: data ingestion
  - Stage 2: landing zone

Main deliverable

- It has to implement:
  - Architecture design
  - Stage 0: project context
  - Stage 1: data ingestion
  - Stage 2: landing zone

- The necessary sections are outlines in the document

Constraint 10. Organization. Clearly organize your code and storage repositories per zones / stages. Each zone should have a folder with its respective scripts (e.g. one folder for the data ingestion and another for the Landing Zone). Clearly separate the scripts per task (if / when required).

Follow-up deliverables

- First deliverable:
  - Project context (first version),
  - Architecture design (first version).

- Second deliverable:
  - Project context (updated).
  - Architecture design (updated).
  - Basic implementation of the data ingestion and the landing zone